```
MODULE Overwatch
EXTENDS Naturals, FiniteSets, Sequences, FailurePolicy

CONSTANTS Names, MaxEdits, Timeout
ASSUME "timeout" ∉ Names

Versions ≜ 0 .. MaxEdits
Results ≜ SUBSET Names
TimedOut ≜ "timeout"
Outcomes ≜ SUBSET (Names ∪ {TimedOut})
NoMessage ≜ [held ↦ FALSE, waking ↦ FALSE, failures ↦ {}]
NotTold ≜ [kind ↦ "none", fresh ↦ {}]

VARIABLES
    truth, head, based, busy, from, quiet, installed,
    produced, consumed, pending, report, baseline, wasFailing,
    outbox, told, flush, turn, toldThisTurn, promptQueued, knows, elapsed

vars ≜ ⟨truth, head, based, busy, from, quiet, installed,
        produced, consumed, pending, report, baseline, wasFailing,
        outbox, told, flush, turn, toldThisTurn, promptQueued, knows, elapsed⟩

TypeOK ≜
    ∧ truth ∈ [Versions → Results]
    ∧ head ∈ Versions ∧ based ∈ 0 .. head
    ∧ busy ∈ BOOLEAN ∧ from ∈ Versions ∧ quiet ∈ BOOLEAN ∧ installed ∈ BOOLEAN
    ∧ produced ∈ 0 .. (MaxEdits + 1) ∧ consumed ∈ 0 .. produced
    ∧ pending ∈ Seq([id : 1 .. (MaxEdits + 1), at : Versions, failures : Outcomes])
    ∧ Len(pending) ≤ MaxEdits + 1
    ∧ report ∈ [known : BOOLEAN , id : 0 .. produced, at : Versions, failures : Outcomes]
    ∧ baseline ∈ [known : BOOLEAN , failures : Outcomes]
    ∧ wasFailing ∈ BOOLEAN
    ∧ outbox ∈ [held : BOOLEAN , waking : BOOLEAN , failures : Outcomes]
    ∧ told ∈ [kind : {"none", "failed", "recovered"}, fresh : Outcomes]
    ∧ flush ∈ BOOLEAN
    ∧ turn ∈ {"idle", "running"}
    ∧ toldThisTurn ∈ BOOLEAN ∧ promptQueued ∈ BOOLEAN
    ∧ knows ∈ Outcomes ∧ elapsed ∈ 0 .. Timeout

Failing ≜ report.known ∧ report.failures ≠ {}
Regressed(f) ≜ TimedOut ∈ f ∨ (baseline.known ∧ NewFailure(f, baseline.failures))
Guard ≜ Failing ∧ Regressed(report.failures)

Init ≜
    ∧ truth ∈ [Versions → Results]
    ∧ head = 0 ∧ based = 0
    ∧ busy = TRUE ∧ from = 0 ∧ quiet = FALSE ∧ installed = FALSE
    ∧ produced = 0 ∧ consumed = 0 ∧ pending = ⟨⟩
    ∧ report = [known ↦ FALSE, id ↦ 0, at ↦ 0, failures ↦ {}]
    ∧ baseline = [known ↦ FALSE, failures ↦ {}]
    ∧ wasFailing = FALSE
    ∧ outbox = NoMessage ∧ told = NotTold ∧ flush = FALSE
    ∧ turn = "idle" ∧ toldThisTurn = FALSE ∧ promptQueued = FALSE
    ∧ knows = {} ∧ elapsed = 0

Queue(signature, message) ≜
    IF told = signature ∧ ¬outbox.held
     THEN UNCHANGED ⟨outbox, told, flush⟩
     ELSE ∧ told′ = signature ∧ outbox′ = message ∧ flush′ = TRUE
```

```
Deliver(v, id, f) ≜
    LET b ≜ IF baseline.known THEN baseline.failures ELSE IF head = based THEN f ELSE {}
        isRegression ≜ TimedOut ∈ f ∨ ¬(f ⊆ b)
    IN  ∧ report' = [known ↦ TRUE, id ↦ id, at ↦ v, failures ↦ f]
        ∧ baseline' = IF TimedOut ∈ f THEN baseline ELSE [known ↦ TRUE, failures ↦ b]
        ∧ wasFailing' = isRegression
        ∧ IF isRegression
            THEN Queue([kind ↦ "failed", fresh ↦ f \ b], [held ↦ TRUE, waking ↦ TRUE, failures ↦ f])
          ELSE IF ¬wasFailing
            THEN UNCHANGED ⟨outbox, told, flush⟩
          ELSE IF outbox.held
            THEN ∧ outbox' = NoMessage ∧ told' = NotTold ∧ UNCHANGED flush
          ELSE Queue([kind ↦ "recovered", fresh ↦ {}], [held ↦ TRUE, waking ↦ FALSE, failures ↦ f])

Edit ≜
    ∧ turn = "running" ∧ head < MaxEdits
    ∧ head' = head + 1 ∧ quiet' = TRUE
    ∧ UNCHANGED ⟨truth, based, busy, from, installed, produced, consumed, pending,
                 report, baseline, wasFailing, outbox, told, flush, turn,
                 toldThisTurn, promptQueued, knows, elapsed⟩

HeadMoves ≜
    ∧ head < MaxEdits
    ∧ head' = head + 1 ∧ based' = head + 1
    ∧ baseline' = [known ↦ FALSE, failures ↦ {}]
    ∧ quiet' = TRUE ∧ pending' = ⟨⟩
    ∧ from' = IF busy THEN head + 1 ELSE from
    ∧ elapsed' = IF busy THEN 0 ELSE elapsed
    ∧ UNCHANGED ⟨truth, busy, installed, produced, consumed, report, wasFailing, outbox,
                 told, flush, turn, toldThisTurn, promptQueued, knows⟩

QuietElapses ≜
    ∧ quiet ∧ installed
    ∧ quiet' = FALSE ∧ busy' = TRUE ∧ from' = head ∧ elapsed' = 0
    ∧ UNCHANGED ⟨truth, head, based, installed, produced, consumed, pending,
                 report, baseline, wasFailing, outbox, told, flush, turn,
                 toldThisTurn, promptQueued, knows⟩

Tick ≜
    ∧ busy ∧ elapsed < Timeout ∧ elapsed' = elapsed + 1
    ∧ UNCHANGED ⟨truth, head, based, busy, from, quiet, installed, produced,
                 consumed, pending, report, baseline, wasFailing, outbox,
                 told, flush, turn, toldThisTurn, promptQueued, knows⟩
```

Completion appends a result; it does not deliver it. Partial output is a stuttering step at this abstraction: only a complete build enters pending.

```
Buffer(v, f) ≜
    ∧ produced' = produced + 1
    ∧ pending' = Append(pending, [id ↦ produced + 1, at ↦ v, failures ↦ f])
    ∧ busy' = FALSE ∧ installed' = TRUE
    ∧ UNCHANGED ⟨truth, head, based, from, quiet, consumed, report, baseline,
                 wasFailing, outbox, told, flush, turn, toldThisTurn, promptQueued, knows, elapsed⟩

RunEnds ≜ busy ∧ ∃ v ∈ from .. head : Buffer(v, truth[v])
TimesOut ≜ busy ∧ elapsed = Timeout ∧ Buffer(from, {TimedOut})

ReadResults ≜
    ∧ pending ≠ ⟨⟩
```

```
    ∧ LET latest ≜ pending[Len(pending)]
      IN  Deliver(latest.at, latest.id, latest.failures)
    ∧ consumed′ = produced ∧ pending′ = ⟨⟩
    ∧ UNCHANGED ⟨truth, head, based, busy, from, quiet, installed, produced,
                 turn, toldThisTurn, promptQueued, knows, elapsed⟩

Held ≜ IF turn = "running" THEN toldThisTurn ELSE promptQueued ∨ ¬outbox.waking

Flush ≜
    ∧ flush ∧ flush′ = FALSE
    ∧ IF outbox.held ∧ ¬Held
        THEN ∧ outbox′ = NoMessage ∧ knows′ = outbox.failures
             ∧ IF turn = "running"
                 THEN toldThisTurn′  = TRUE ∧ UNCHANGED promptQueued
                 ELSE promptQueued′ = TRUE ∧ UNCHANGED toldThisTurn
        ELSE UNCHANGED ⟨outbox, knows, toldThisTurn, promptQueued⟩
    ∧ UNCHANGED ⟨truth, head, based, busy, from, quiet, installed, produced, consumed,
                 pending, report, baseline, wasFailing, told, turn, elapsed⟩

TurnStarts ≜
    ∧ turn = "idle" ∧ turn′ = "running"
    ∧ toldThisTurn′ = promptQueued ∧ promptQueued′ = FALSE ∧ flush′ = TRUE
    ∧ UNCHANGED ⟨truth, head, based, busy, from, quiet, installed, produced, consumed,
                 pending, report, baseline, wasFailing, outbox, told, knows, elapsed⟩

WokenTurnStarts ≜ promptQueued ∧ TurnStarts

TurnCompletes ≜
    ∧ turn = "running"
    ∧ IF Guard
        THEN ∧ toldThisTurn′ = TRUE ∧ knows′ = report.failures ∧ UNCHANGED ⟨turn, flush⟩
        ELSE ∧ turn′ = "idle" ∧ flush′ = TRUE ∧ UNCHANGED ⟨toldThisTurn, knows⟩
    ∧ UNCHANGED ⟨truth, head, based, busy, from, quiet, installed, produced, consumed,
                 pending, report, baseline, wasFailing, outbox, told, promptQueued, elapsed⟩

Next ≜ Edit ∨ HeadMoves ∨ QuietElapses ∨ Tick ∨ RunEnds ∨ TimesOut
       ∨ ReadResults ∨ Flush ∨ TurnStarts ∨ TurnCompletes

Spec ≜
    ∧ Init ∧ □[Next]_vars
    ∧ WF_vars(QuietElapses) ∧ WF_vars(Tick) ∧ WF_vars(TimesOut)
    ∧ WF_vars(ReadResults) ∧ WF_vars(Flush)
    ∧ WF_vars(WokenTurnStarts) ∧ WF_vars(TurnCompletes)

BufferedResultsAreOrdered ≜
    ∧ ∀ i ∈ 1 .. Len(pending) : pending[i].id > consumed ∧ pending[i].id ≤ produced
    ∧ ∀ i ∈ 1 .. (Len(pending) − 1) : pending[i].id < pending[i + 1].id

ReadsSelectLatestComplete ≜ report.known ⇒ report.id = consumed
ResultsMatchBuilds ≜ report.known ⇒ report.failures = truth[report.at] ∨ report.failures = {TimedOut}
BaselineNeverAbsorbsEdits ≜ baseline.known ⇒ baseline.failures ⊆ truth[based]
RunsEnd ≜ busy ⇝ ¬busy
EveryEditIsChecked ≜ ◇□(report.known ∧ report.at = head)
RegressionsReachAgent ≜ ◇□((truth[head] \ truth[based]) ⊆ knows ∨ TimedOut ∈ knows)
```